EE 250L: Distributed Systems for the Internet of Things

Fall 2026 | Lab 7

PCB Design for the ESP32-S3 Sensor Board

	Tuesday Lab	Thursday Lab
Assigned Date	10/20/26	10/22/26
Submission HARD Due Date	10/26/26 (3 PM)	10/28/26 (3 PM)

Video tutorials (watch these first): USC EE 250: ESP32-S3 Lab Sensor Board (YouTube playlist)

Teams:

This lab is done in **teams of two**. Each team submits **one** layout, named with your lab section, one partner's last four USC ID digits and that partner's last name; the same digits and last name go on the silkscreen. Both partners are interviewed individually and must be able to open and explain the layout.

Files you need: `sensor_lab_files.zip` (symbols, footprints and 3D models for this lab).

File to submit: **only one submission per group** for your layout file, renamed `TU_####_Lastname.kicad_pcb` (Tuesday section) or `TH_####_Lastname.kicad_pcb` (Thursday section), where `####` is the last four digits of one partner's USC ID and `Lastname` is that partner's last name, uploaded to the Google submission form. See Section 6.

1. Overview

In Lab 6 you built five sensor and actuator circuits on a breadboard and wired them to an ESP32-S3 development board: a light sensor (photoresistor), a temperature sensor (NTC thermistor), a vibration switch, a push button and an LED. You proved, in firmware, that each branch works. In this lab you turn that tested prototype into a permanent printed circuit board (PCB).

The board you design is a $2.50'' \times 2.50''$ carrier: the ESP32-S3 dev board (**U1**, YD-ESP32-S3) plugs into two 1×22 female headers on the right-hand side, and the five sensor branches sit to its left, connected by copper traces instead of jumper wires (Figure 1). Because the ESP32-S3 has a 12-bit ADC on-chip, there is no external ADC chip and no ribbon cable — the analog sensors connect straight to the dev board's GPIO pins.

All submitted layouts will be combined (panelized) into a few large boards, manufactured and cut apart. You will assemble your own board in a later lab, so design it as if you are going to solder it.

2. Design Requirements

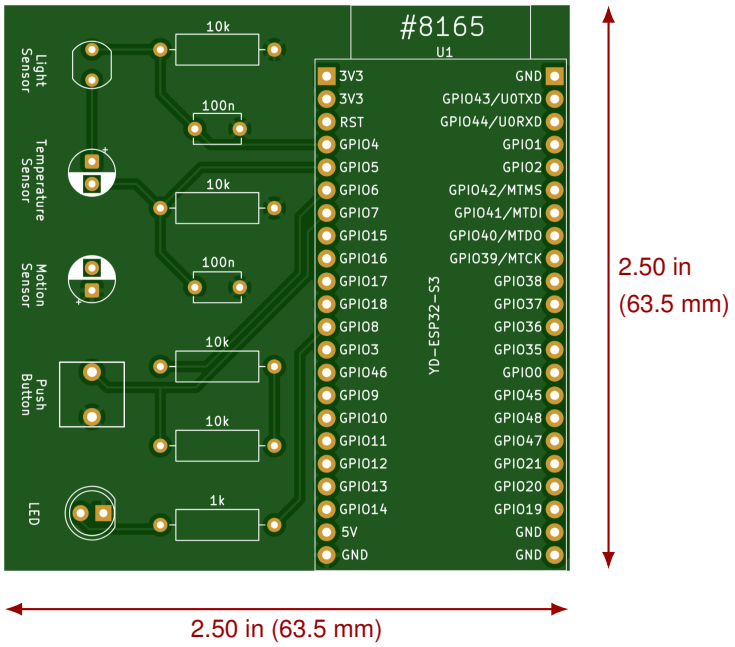


Figure 1: The reference layout (top side). Sensor branches on the left, ESP32-S3 dev board footprint (U1) on the right, and the designer's USC ID digits on the front silkscreen (#8165 in this example; yours will also carry a last name, e.g. `#8165 SMITH`).

Every submission must meet all four requirements.

1. **Board size.** The board outline (a rectangle on the **Edge.Cuts** layer) must measure **exactly** $\mathbf{2.50''} \times \mathbf{2.50''}$ ($63.5\,\text{mm} \times 63.5\,\text{mm}$).
2. **Provided libraries.** You must use the symbol and footprint files included in `sensor_lab_files.zip`. Do not substitute other ESP32 or push-button footprints from SnapEDA, the KiCad library or elsewhere — the panel only works if every board uses the same parts.
3. **Your ID on the board.** Your layout must include the **last four digits of one partner's USC student ID and that partner's last name** (e.g. `#8165 SMITH`) as text on either the front (**F.Silkscreen**) or back (**B.Silkscreen**) silkscreen layer.
4. **What and how to submit.** Submit **only** your layout file, named `TU_####_Lastname.kicad_pcb` (Tuesday section) or `TH_####_Lastname.kicad_pcb` (Thursday section), where `####` is the last four digits of **one** partner's USC ID and `Lastname` is that partner's last name (e.g. `TU_8165_Smith.kicad_pcb`). Submit once per group; the same four digits and last name must also appear on the silkscreen.

Layouts that are the wrong size, use other footprints, or are missing the ID cannot be placed on the panel, so please check these before you submit (Section 6 has a checklist).

3. Setting Up KiCad

Follow along with the video tutorials. The steps below follow the tutorial playlist (link). Watch each video before doing its section:

Video	Topic	Manual section
1	Lab overview, the sensor circuit, downloading and installing KiCad	3.1
2	What is inside `sensor_lab_files.zip`	3.3
3	Creating the project and adding the provided libraries	3.4
4	Drawing the schematic	4
5	Annotation, ERC and footprint assignment	4.4
6	PCB Editor board setup (design rules)	5.1
7	Board layers and the $2.50'' \times 2.50''$ outline	5.3
8	Importing and placing footprints, silkscreen labels	5.5–5.6
9	Routing traces and vias	5.7
10–11	DRC, ID and silkscreen border, optional logo, submitting	5.8–6
12	Generating Gerber files (optional, not submitted)	Appendix A

3.1 Download and installation

1. Go to the official KiCad website, `https://kicad.org/download/`, and choose your operating system (Windows, macOS or Linux — KiCad is cross-platform, and your layout file will work no matter which one you use).
2. Download and install the **latest stable release, KiCad 10**. The provided library files were saved with KiCad 10, and older versions (KiCad 9 and earlier) cannot open them.
3. Launch KiCad from your applications or start menu.

3.2 Overview of KiCad's key tools

In the KiCad Project Manager window you will find the tools used in this lab:

- **Schematic Editor**: draw the circuit by placing symbols and connecting them with wires.
- **Symbol Editor**: create, edit or import schematic symbols.
- **PCB Editor**: place the physical footprints and route the copper traces.
- **Footprint Editor**: create, edit or import footprints (pad sizes and positions).
- **3D Viewer**: preview the assembled board (opened from the PCB Editor).
- **Gerber Viewer**: inspect manufacturing (Gerber) files.
- **Image Converter**: turn an image into a silkscreen footprint (optional, for artwork and logos).

3.3 The provided files

Download `sensor_lab_files.zip` (Google drive) and extract it. It has two folders (Table 1):

- `components/` — the third-party parts that are **not** in KiCad's built-in libraries: a symbol and a footprint for the ESP32-S3 dev board, and a custom **footprint only** for the 6 mm two-pin push button from your kit. (The button uses KiCad's built-in push-button symbol, so it needs no symbol file.) The `.step`/`.stp` files are 3D models; they are only needed for the 3D Viewer or for exporting the board to

a mechanical CAD tool.

- `documentation/` — the reference schematic drawing and screenshots of the settings you will enter in KiCad (footprint assignments, design-rule constraints, pre-defined sizes).

Folder	File	Purpose
`components/YD-ESP32-S3/`	`YD-ESP32-S3.kicad_sym`	ESP32-S3 dev board symbol (44 pins)
	`YD-ESP32-S3.kicad_mod`	Footprint (two 1×22 pad rows)
	`YD-ESP32-S3.step`	3D model
`components/SW_PUSH_6mm_2pin/`	`SW_PUSH_6mm_2pin.kicad_mod`	Footprint for the 6 mm two-pin push button
	`SW_PUSH_6mm_2pin.stp`	3D model
`documentation/`	`schematic.png`	Reference schematic drawing
	`footprint_assignments.png`	Footprint for every part (Section 4.4)
	`drc_mm.png`	Design-rule constraints (Section 5.1)
	`predefined_sizes.png`	Track and via sizes (Section 5.1)

Table 1: Contents of `sensor_lab_files.zip`. A `.kicad_sym` file is a symbol, a `.kicad_mod` file is a footprint, and a `.step`/`.stp` file is a 3D model.

3.4 Create the project and add the provided libraries

1. In the Project Manager, click **New Project**. Keep the default option, pick a location, make sure **Create a new folder for the project** is checked, name the project (e.g. `ee250_sensor_lab`) and click **Save**. The folder contains three important files: the project file `.kicad_pro` ("the glue that holds everything together" — always reopen your work from this file), the schematic `.kicad_sch`, and the layout `.kicad_pcb`, which is the file you will submit. KiCad also auto-saves into a `backups` folder.
2. Move the `components` and `documentation` folders from the ZIP **into your project folder**, so everything for this lab lives in one place.
3. **Add the symbol.** Open the **Schematic Editor**, go to **Preferences** → **Manage Symbol Libraries**, choose the **Project Specific Libraries** tab (not Global), click the folder icon (*Add existing*) and select `components/YD-ESP32-S3/YD-ESP32-S3.kicad_sym`. Keep the nickname `YD-ESP32-S3` and click OK. Only the ESP32 needs a symbol.
4. **Add the footprints.** Switch to the **PCB Editor**, go to **Preferences** → **Manage Footprint Libraries** → **Project Specific Libraries**, and click the folder icon. Here KiCad asks for the **folder that contains** the footprint files, not the `.kicad_mod` file itself: select the `components/YD-ESP32-S3` folder, then repeat for `components/SW_PUSH_6mm_2pin`. Click OK.

Why project-specific libraries?. Adding third-party parts to the *project* library rather than the global one keeps them with this project, makes them easy to find, and lets you modify them for this project without changing KiCad's built-in libraries.

4. Schematic Design

Open the **Schematic Editor**. Your schematic must contain the same circuit you tested in Lab 6, now connected directly to the ESP32-S3 symbol. It must be **topologically equivalent** to Figure 2; the exact positions of the symbols do not matter.

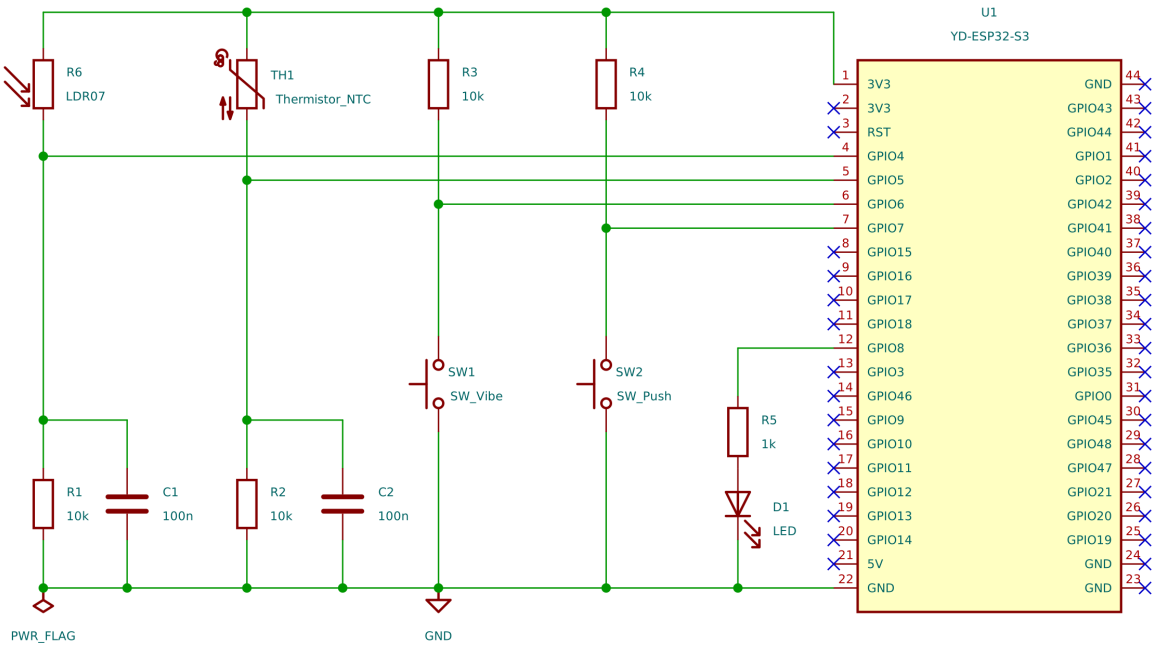


Figure 2: Reference schematic. Five branches between 3V3 and GND, each tapped to one GPIO of U1: light sensor → GPIO4, temperature sensor → GPIO5, vibration switch → GPIO6, push button → GPIO7, LED ← GPIO8. Unused ESP32 pins carry no-connect flags (blue ×).

Branch	Parts	Circuit	GPIO
Light sensor	LDR (GL5549), 10 kΩ, 100 nF	3V3 → LDR → node → 10 kΩ → GND	GPIO4 (ADC1_CH3)
Temperature	NTC 10 kΩ, 10 kΩ, 100 nF	3V3 → NTC → node → 10 kΩ → GND	GPIO5 (ADC1_CH4)
Vibration	10 kΩ, vibration switch	3V3 → 10 kΩ → node → switch → GND	GPIO6
Push button	10 kΩ, push button	3V3 → 10 kΩ → node → button → GND	GPIO7
LED	1 kΩ, 5 mm red LED	GPIO8 → 1 kΩ → LED anode; cathode → GND	GPIO8

Table 2: The five branches (same circuit as Lab 6). In each analog branch the 100 nF capacitor sits in parallel with the 10 kΩ resistor to filter noise. 3V3 comes from U1 pin 1 and GND from U1 pin 22.

Reading the reference drawing in `documentation/schematic.png`. In that drawing the GPIO5, GPIO6 and GPIO7 lines are routed near the ground symbols, which can look as if the GPIOs connect to ground. They do not: each GPIO connects only to the **dot labeled with that GPIO name** in its branch (the divider node or the pull-up node). Figure 2 shows the actual connections.

4.1 Useful hotkeys

Key	Action	Key	Action
A	Place symbol	W	Draw wire
P	Place power symbol	Q	No-connect flag
E	Properties of the selected item	R	Rotate
M	Move	Esc	Back to the normal cursor
Ctrl+C/Ctrl+V	Copy / paste (references auto-increment)	Ctrl+S	Save (Cmd on macOS)

Every hotkey is also listed in the right-click menus. If you learn only one, learn E (properties).

4.2 Placing the symbols

Press A to open the **Choose Symbol** window and search for each part. Double-click a part to place it, then press Esc. To change a value, select the symbol, press E and edit the **Value** field.

- **U1 (ESP32-S3 dev board)**: type YD and pick YD-ESP32-S3 — the symbol you added to the project library.
- **LDR**: type LDR and pick LDR07 (under optical sensors). You may set its value to GL5549, the part in your kit.
- **Thermistor**: type thermistor and pick the **NTC** thermistor (Thermistor_NTC).
- **Vibration switch**: KiCad has no vibration-switch symbol, so use the generic two-pin push-button symbol (SW_Push). Rotate it with R to match the others and set its value to SW_Vibe.
- **Push button**: copy and paste the switch, and set the copy's value to SW_Push.
- **LED**: the generic LED symbol.
- **Resistors**: type res and pick R. You need **four 10k** and **one 1k** resistor.
- **Capacitors**: the unpolarized C symbol (ceramic capacitors). You need **two**, value 100n.
- **Ground**: press P, search GND, and place **one** ground symbol — every branch shares a common ground.
- **Power flag**: press P, search PWR, and place a PWR_FLAG on the ground node. It tells the ERC that this net is driven, because the circuit is powered from the dev board's 3.3 V relative to this ground.

4.3 Wiring

Press W and click from pin to pin. A green dot (**junction**) means wires are joined into one node; where a wire simply crosses another there must be **no** dot.

1. Connect the top of every branch together and to U1's **3V3** pin (pin 1).
2. Connect the bottom of every branch to the common ground node, and that node to U1's **GND** pin.
3. Connect each GPIO to its branch: **GPIO4** to the LDR divider node, **GPIO5** to the thermistor divider node, **GPIO6** to the vibration-switch pull-up node, **GPIO7** to the push-button pull-up node, and **GPIO8** (pin 12) to the 1k resistor feeding the LED's anode. The LED's cathode goes to ground.
4. Watch out when a GPIO wire passes over other branches: if it lands on a wire it will join that node. If that happens, press Esc, move the branches out of the way, and draw the wire again.
5. Place a **no-connect flag** (Q) on every unused U1 pin. The small open circle on a pin disappears once the flag is placed. (Tip: place a few flags, then copy and paste the group.)

Save often with Ctrl+S.

4.4 Annotate, ERC and footprint assignment

Do these in order, using the buttons on the top toolbar from left to right:

1. **Annotate Schematic.** Click it, leave the default options, click **Annotate**, then **Close**. Every symbol gets a unique reference (R1, R2, C1, . . .) that carries over to the layout. Your numbering may differ from the reference design.
2. **Run the ERC** (Electrical Rules Checker). You want **0 errors**. At this stage you will likely see **one warning** about the ESP32 symbol's footprint library; it goes away once you assign footprints in the next step. Click **Delete All Markers** to clear the warning arrow. Fix any errors before you continue.
3. **Assign Footprints.** Click it and assign a footprint to every reference, following Table 3 and `documentation/footprint_assignments.png` (Figure 3). Pick a library on the left, type part of the name in the filter box at the top, and double-click a footprint on the right to assign it to the highlighted reference; KiCad then moves to the next one. Clear the filter box when you switch libraries.
4. Click **Apply, Save Schematic & Continue**, then **OK**.
5. **Re-run the ERC.** You should now get **0 errors and 0 warnings**.

Part	Library	Footprint
Capacitors (100n)	`Capacitor_THT`	`C_Disc_D5.1mm_W3.2mm_P5.00mm`
LED	`LED_THT`	`LED_D5.0mm`
Resistors (10k and 1k)	`Resistor_THT`	`R_Axial_DIN0309_L9.0mm_D3.2mm_P12.70mm_Horizontal`
LDR	`OptoDevice`	`R_LDR_5.1x4.3mm_P3.4mm_Vertical` (usually pre-filled)
Vibration switch	`Capacitor_THT`	`CP_Radial_D5.0mm_P2.50mm`
NTC thermistor	`Capacitor_THT`	`CP_Radial_D5.0mm_P2.50mm`
Push button	`SW_PUSH_6mm_2pin` *(provided)*	`SW_PUSH_6mm_2pin`
ESP32-S3 dev board	`YD-ESP32-S3` *(provided)*	`YD-ESP32-S3`

Table 3: Footprint assignments. All parts are through-hole (THT). The vibration switch looks like a small electrolytic capacitor, so it and the thermistor use the 5 mm radial capacitor footprint with 2.5 mm lead spacing. The resistors in your kit have long leads, so the 12.7 mm axial footprint fits them comfortably.

This completes the schematic. Before moving on, check that the topology is right, that no wire accidentally formed a junction, and that you have one common ground node with a power flag.

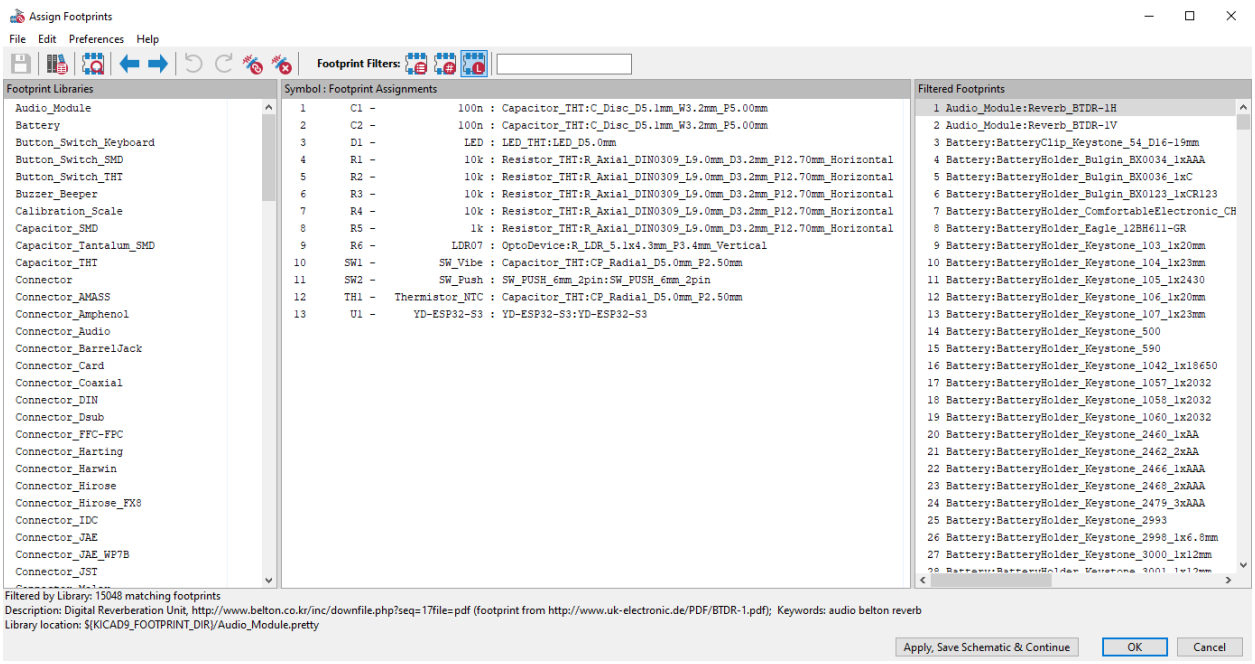


Figure 3: The Assign Footprints window with all 13 footprints assigned (`documentation/footprint_assignments.png`).

5. PCB Layout and Routing

Click **Switch to PCB Editor** on the schematic toolbar. Unlike the schematic, the layout has real physical dimensions.

5.1 Board setup: design rules

Before placing anything, set the design rules that the **Design Rules Checker (DRC)** will enforce, so that a manufacturer can actually produce your board. Open **Board Setup** (toolbar button, or **File** → **Board Setup**). This is a two-layer board, so the stack-up defaults are fine. Work in **millimeters**; the units button on the left toolbar switches between inches, mils and mm.

Design Rules → **Constraints** (Figure 4):

Setting	Value	Setting	Value
Minimum clearance	0 mm	Copper to edge clearance	0.25 mm
Minimum track width	**0.25 mm**	Minimum drill size	0.25 mm
Minimum connection width	0 mm	Hole to hole clearance	0.25 mm
Minimum annular width	0.1 mm	Minimum uVia diameter / hole	0.2 / 0.1 mm (not used)
Minimum via diameter	0.5 mm	Minimum silk text height	0.5 mm
Copper to hole clearance	0.25 mm	Minimum silk text thickness	0.1 mm

Note: `drc_mm.png` shows a minimum track width of 0.2 mm; use **0.25 mm** as in the video. Micro-vias greatly increase manufacturing cost and are not used in this lab.

Design Rules → **Pre-defined Sizes** (Figure 5): add a **0.25 mm** track width and a via with **0.8 mm** diameter and **0.4 mm** hole (the default via is a bit small). Click **OK**.

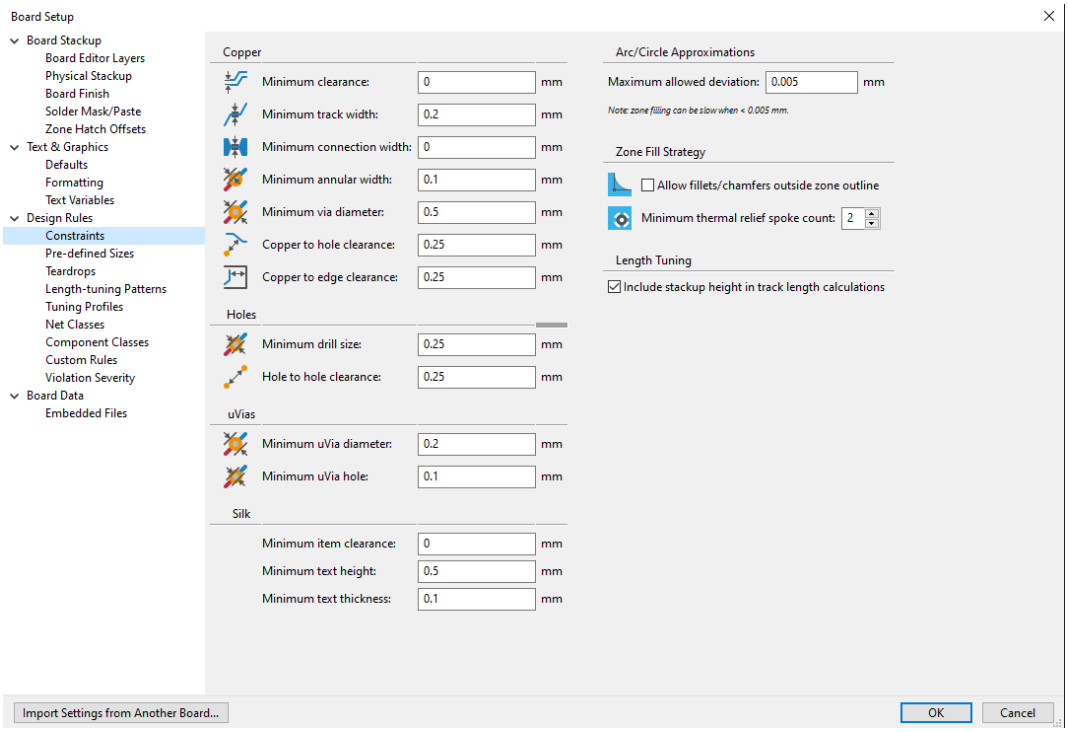


Figure 4: Board Setup → Design Rules → Constraints, from `documentation/drc_mm.png`. Set the minimum track width to 0.25 mm rather than the 0.2 mm shown.

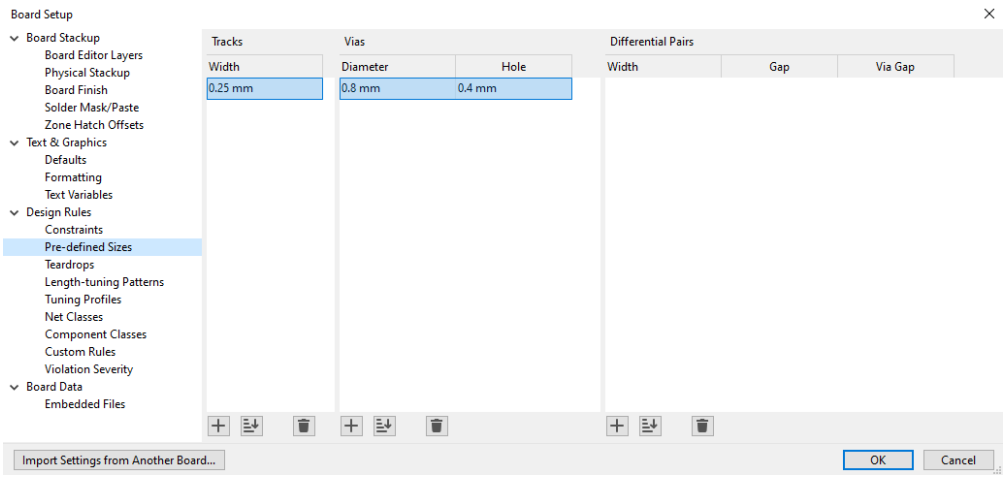


Figure 5: Board Setup → Design Rules → Pre-defined Sizes: track width 0.25 mm, via 0.8 mm diameter with a 0.4 mm hole.

Tip: **Import Settings from Another Board** at the bottom of Board Setup copies all of these rules from a saved project — handy for your future PCB projects.

5.2 PCB layers

The **Layers** panel on the right lists every layer. The ones that matter here:

- **F.Cu** / **B.Cu**: front (top) and back (bottom) copper. Most signal traces run on the front, and some run on the back so they can cross each other.

- **F.Silkscreen** / **B.Silkscreen**: the white letters, labels and images printed on the board.
- **F.Mask** / **B.Mask**: solder mask — defines which copper is exposed (pads) and which is covered by the thin insulating mask.
- **F.Paste** / **B.Paste**: solder paste for automated assembly (leave as is). **F.Adhesive** / **B.Adhesive**: not used.
- **F.Courtyard**: the outline around each footprint (keep courtyards from overlapping). **F.Fab**: fabrication/comment layer — the gray reference and value text. Click its eye icon to hide it when the view gets cluttered.
- **Edge.Cuts**: the board outline — defines the physical size and shape of your PCB.

5.3 Drawing the board outline (2.50 in $\times$ 2.50 in)

Draw the outline **before** importing the footprints, so you have a boundary to place them in.

1. Select the **Edge.Cuts** layer and click **Draw Rectangle**. Click once anywhere, move the mouse, click again, then press `Esc`.
2. Select the rectangle and press `E`. Switch the units to **inches** and enter a size of **2.5** for both X and Y (or 63.5 in millimeters). Where the board sits on the canvas does not matter; only the size does.
3. Click OK. Do not size the outline by dragging it on the grid — it must be exactly $2.50'' \times 2.50''$ (Requirement 1).

The **3D Viewer** (toolbar button, or **View** $\rightarrow$ **3D Viewer**) now shows a blank two-layer board. Use it often as you work.

5.4 Importing the footprints

1. Save the schematic, then click **Update PCB from Schematic** (`F8`) and click **Update PCB**, then **Close**.
2. All footprints are now attached to your cursor; click anywhere to drop them, then press `Esc`.

Note: the Update PCB from Schematic button only appears if you opened the PCB Editor from the **project**. Always reopen your work by double-clicking the `.kicad_pro` file, not the `.kicad_pcb` file.

The thin teal lines between pads are the **ratsnest**. They are not traces — they show which pads still need to be connected. Your goal while placing is to untangle them as much as possible.

5.5 Placing the components

1. **U1 first.** Only pins on the left row of U1 are used, so move U1 to the **right side** of the board. To center it vertically, select U1, `Ctrl`-click the Edge.Cuts rectangle, right-click and choose **Align/Distribute** $\rightarrow$ **Align to Vertical Center**. The footprint's pins are labeled, so the dev board's orientation is unambiguous. You will not solder the dev board directly: you will solder two 1×22 **female headers** into these pads and plug the dev board in, so it can be removed.
2. **The five sensors.** As on your breadboard, place the sensors in a column on the left, top to bottom: light sensor (LDR), temperature sensor (thermistor), motion sensor (vibration switch), push button, LED. Press `R` to rotate a footprint so its pads face the parts they connect to.
3. **Resistors and capacitors.** Put each branch's resistor (and capacitor, for the two analog branches) between its sensor and U1. To find a part, select its symbol in the Schematic Editor: the matching footprint is highlighted in the PCB Editor (handy with two monitors).
4. **Align.** Select a column of parts, right-click and choose **Align/Distribute** $\rightarrow$ **Align to Horizontal Center**, then **Distribute Vertically** to space them evenly. Do the same for the sensor column.

Everything fits on the $2.50'' \times 2.50''$ board; Figure 1 shows the reference arrangement, but yours does not have to match.

5.6 Silkscreen: labels and your ID

1. **Branch labels.** Select the **F.Silkscreen** layer and click the **Text** tool (**T** icon). In millimeters, set width and height to **1 mm** and thickness to **0.15 mm**, type e.g. "Light", press Enter, type "Sensor", and set the justification to center. Rotate it with `R` and place it next to the sensor. Copy and paste it for "Temperature Sensor", "Motion Sensor", "Push Button" and "LED", then align the labels (**Align to Horizontal Center**).
2. **Tidy the designators.** Select a footprint, press `E`, and uncheck **Show** for its **Reference** (R1, SW2, ...) so it no longer prints. For the resistors and capacitors, instead show the **Value** (10k, 1k, 100n) and change its layer from F.Fab to **F.Silkscreen**, so the values are printed on the board — you will need them when you solder.

You will add your ID and a silkscreen border after routing (Section 5.9). Check the 3D Viewer: every sensor should be labeled and every resistor and capacitor should show its value.

5.7 Routing traces

The goal now is to replace every thin ratsnest line with a copper trace.

1. **Select your sizes.** In the track-width drop-down on the top toolbar, choose your pre-defined **0.25 mm** track instead of the default; in the via drop-down, choose the **0.8/0.4 mm** via.
2. **Route on the front.** Select the **F.Cu** layer (hide F.Fab to declutter), click **Route Single Track** (`X`) and click a pad. KiCad highlights only the pads that pad must connect to, so you do not need to keep checking the schematic. Click to finish the trace and press `Esc`. You can then select a trace and drag it, e.g. to move it away from a nearby pad. Front traces are drawn in red.
3. **Use the back to cross.** When a front trace is in the way, do not route all the way around the board — keep traces short. Select **B.Cu** and route the connection underneath (back traces are drawn in blue).
4. **Rotate parts to shorten paths.** If a connection is awkward, try rotating a footprint (`R`, e.g. twice) and re-aligning it; this often opens a direct path.
5. **Vias.** A via is a plated hole with a copper pad on each side that joins a front trace to a back trace. Click **Add Vias** (make sure your pre-defined via size is selected) and place it. A new via has **no net**: select it, press `E`, and assign the net it belongs to (e.g. `GND`). Then route to it on one layer and away from it on the other.
6. **Check every net.** Keep going until no ratsnest lines remain. Check the 3.3 V and ground nets in particular — it is easy to miss one last connection. The 3D Viewer shows the copper on both sides.

The reference design also fills both copper layers with a **GND copper zone** (**Add Filled Zone**, net `GND`), which connects the ground pads without separate traces (press `B` to refill zones). This is optional.

5.8 Design Rules Check (DRC)

Click the **DRC** button on the top toolbar (or **Inspect** $\rightarrow$ **Design Rules Checker**). Keep **Refill all zones before performing DRC** checked and click **Run DRC**. You want **0 violations** (errors) and **0 unconnected items**. Warnings matter much less than errors, and the ignored tests can be ignored. Fix any errors and re-run until they are gone.

5.9 Finishing touches: your ID and the silkscreen border

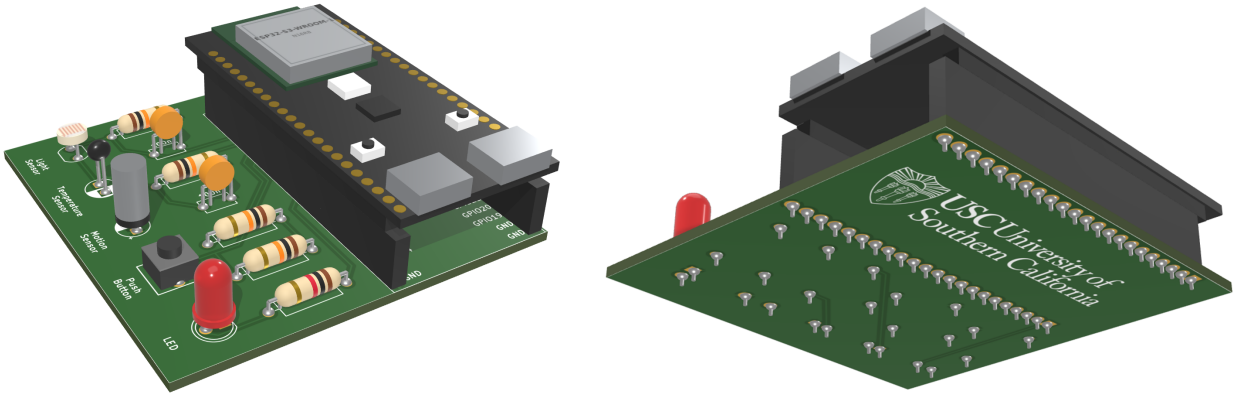
1. **Your ID (Requirement 3).** Select **F.Silkscreen** (or B.Silkscreen) and the **Text** tool, and type a pound sign, the **last four digits of one partner's USC ID**, a space, and that partner's **last name**, e.g. `#1234 SMITH` — but use *your* digits and name, the same ones as in your file name. (The video types only `#8165`; add the last name.) Make it larger than the labels: **2 mm** tall, with a bold stroke. A good spot is the empty area above U1.
2. **Silkscreen border.** All boards are panelized by hand and cut apart, and a border shows where your design ends and the next one begins. Select the **Edge.Cuts** rectangle, copy and paste it (`Ctrl+C`, `Ctrl+V`) exactly on top of the original, and while it is still selected press `E`: change its layer to **F.Silkscreen** and make the line about **10× thicker** (e.g. 0.5 mm). Do the same again with layer **B.Silkscreen**. The Edge.Cuts rectangle stays as it is.
3. **Re-run the DRC.** It should still show 0 errors and 0 unconnected items. You will now see a few **warnings** that the silkscreen border is clipped by the board edge: half of the thick line falls outside the cut. This is expected and fine — the half inside the board is what matters. Click **Delete All Markers**, close the window, and **save**.

5.10 Adding a logo (optional)

You can put artwork on the silkscreen with the **Image Converter** (in the Project Manager). Load a PNG, choose the **Front Silkscreen** output, set the output size (keep the aspect ratio locked; the reference logo is 2 in long), and click **Export to Clipboard**. Paste it into the PCB Editor with `Ctrl+V`, delete its small reference text, rotate it as needed, then press `E` and change its layer to **B.Silkscreen** *and* its side from front to **back**, so it reads correctly from the bottom. Silkscreen is one color, so the image is converted to black and white.

5.11 3D Viewer

Before submitting, carefully inspect the board in the 3D Viewer: part placement, orientation, labels, your ID and the border (Figure 6). If U1 shows only bare pads, open U1's footprint properties, go to **3D Models**, and point the model to `components/YD-ESP32-S3/YD-ESP32-S3.step`.



(a) Top (b) Bottom

Figure 6: 3D rendering of the reference board, with the ESP32-S3 dev board plugged into the two 1×22 female headers. (a) Top: the five sensor branches and their resistors and capacitors. (b) Bottom: solder joints, back-side traces and the USC logo on B.Silkscreen.

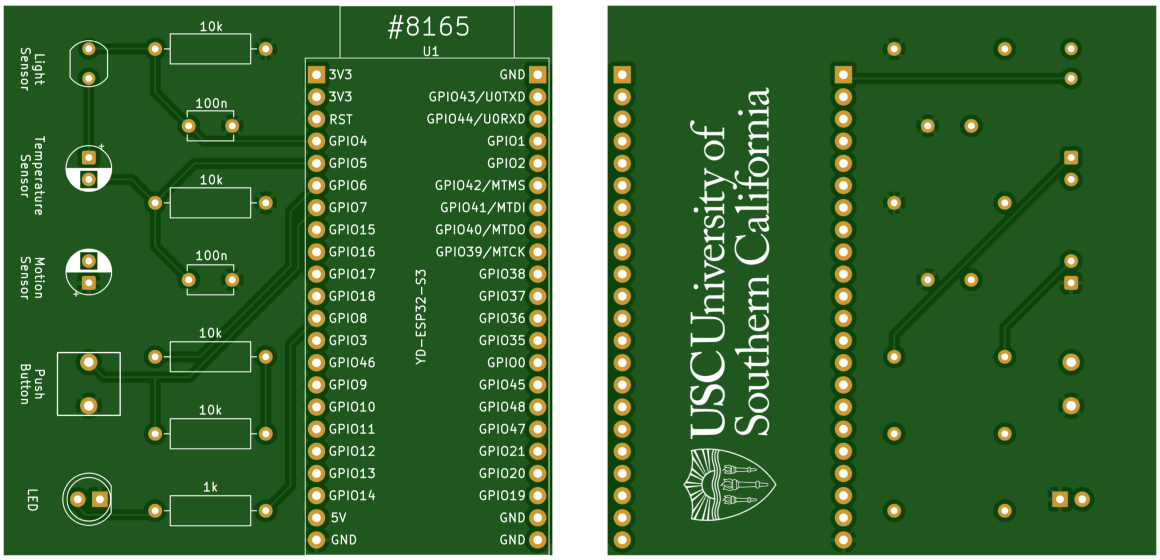


Figure 7: The reference board as it will be manufactured, rendered from its Gerber files. Left: top (front). Right: bottom (back), with the USC logo on B.Silkscreen. Both copper layers carry a GND fill.

6. Submission

You will receive full credit for this lab if your layout meets the four requirements and passes both **ERC** and **DRC** with no errors.

Pre-submission checklist.

- ☐ ERC passes with 0 errors.
- ☐ DRC passes with 0 errors and 0 unconnected items.
- ☐ The Edge.Cuts rectangle is exactly $2.50'' \times 2.50''$ (63.5 mm $\times$ 63.5 mm) — double-click it to check.
- ☐ U1 and the push button use the footprints from `sensor_lab_files.zip`.
- ☐ The last four digits of one partner's USC ID and that partner's last name are on F.Silkscreen or B.Silkscreen.
- ☐ A thick silkscreen border traces the board outline on F.Silkscreen and B.Silkscreen.
- ☐ The submitted file is `TU_####_Lastname.kicad_pcb` or `TH_####_Lastname.kicad_pcb` with your real section, last four ID digits and last name (one per group), and the same digits and last name appear on the silkscreen.

To prepare the file, make a **copy** of your project's `.kicad_pcb` file and rename the copy to your section, four digits and last name (e.g. `TU_8165_Smith.kicad_pcb`); use your *own* section, digits and name, not the example's. Upload **only** that file to the Google submission form by the deadline for your lab section (Mon 10/26 or Wed 10/28, 3 PM). Do not submit the schematic, project file, Gerbers or a ZIP — the TAs combine the layout files into the manufacturing panel and generate the Gerbers themselves.

Component	Points
Demo: ERC with 0 errors	5
Demo: full schematic, five branches on GPIO4–GPIO8	5
Demo: 3D view (top and bottom) with ID digits, last name and silkscreen border	5
Demo: DRC on the submitted file, 0 errors and 0 unconnected items	10
Submission: correct file and name, once per team, on time	5
Interview	70
Total	**100**

A. Generating Gerber files (optional)

You do **not** submit Gerbers for this lab — the TAs build the panel from your layout file. These steps are for when you want a manufacturer to make your own boards in the future.

1. Open the project file, then the PCB Editor, and run the DRC.
2. **File** → **Fabrication Outputs** → **Gerbers**. Set the output directory to a new folder (e.g. `gerbers`) and accept the relative path.
3. Include these layers: **F.Cu, B.Cu, F.Paste, B.Paste, F.Silkscreen, B.Silkscreen, F.Mask, B.Mask, Edge.Cuts**. Leave adhesive, user, comment and fab layers unchecked.
4. Keep **Check zone fills before plotting**, the 4.6 mm coordinate format, the extended X2 format and net-list attributes. Uncheck **Generate Gerber job file** (manufacturers don't use it). Leave *Disable aperture macros* unchecked. Click **Plot**.
5. Click **Generate Drill Files**: Excellon format, absolute origin, millimeters, decimal zeros, no map file. Click **Generate**, then save the board.
6. The folder now holds 9 Gerber files and 2 drill files. Zip the folder — that ZIP is what you upload to a manufacturer.
7. Check it first: open the **Gerber Viewer**, choose **File** → **Open Zip Archive File**, and inspect each layer (e.g. measure the Edge.Cuts outline: 63.5 mm = 2.50 in).